

【专栏一：AI基础06】 - 【一张图讲清楚：大模型怎么知道有哪些工具？又怎么选中正确工具？】



快乐飞猪ing



· 码龄7年 · 粉丝可见 · 编辑



墨衍会员 · AI 创作全网分发

原创 · 2026-03-27 11:55:33 · 425 阅读



AI 写同款 · GEO 优化

【WorkBuddy 专栏】完结篇暨付费专栏总目录（内容继续更新中）

[阅读详情 >](#)

【WorkBuddy 专栏】完结篇和付费内容总目录

前言

上一篇文章，我梳理了 Function Call 的完整流程：

用户提问 → 工具说明输入给 LLM → LLM 判断是否调用工具 → 系统执行工具 → 结果回填给 LLM → 生成最终回答。

但写完之后，我发现还有一个更关键的问题：

- 大模型到底是怎么知道有哪些工具可以调用的？
- 如果系统同时给了它很多工具，它又是怎么选中正确工具的？

这篇文章，我就想专门把这个问题讲清楚。

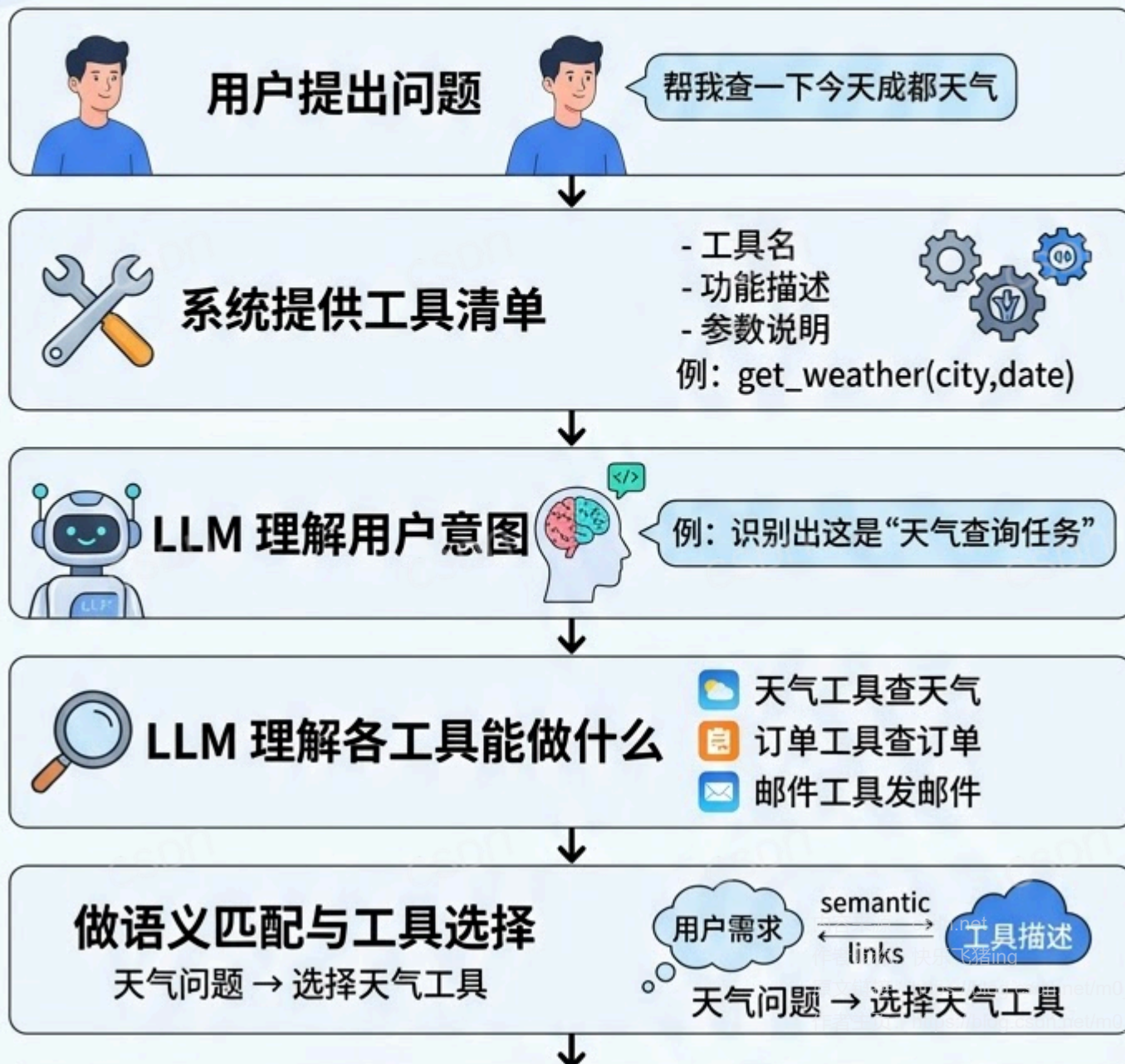
内容来源：csdn.net

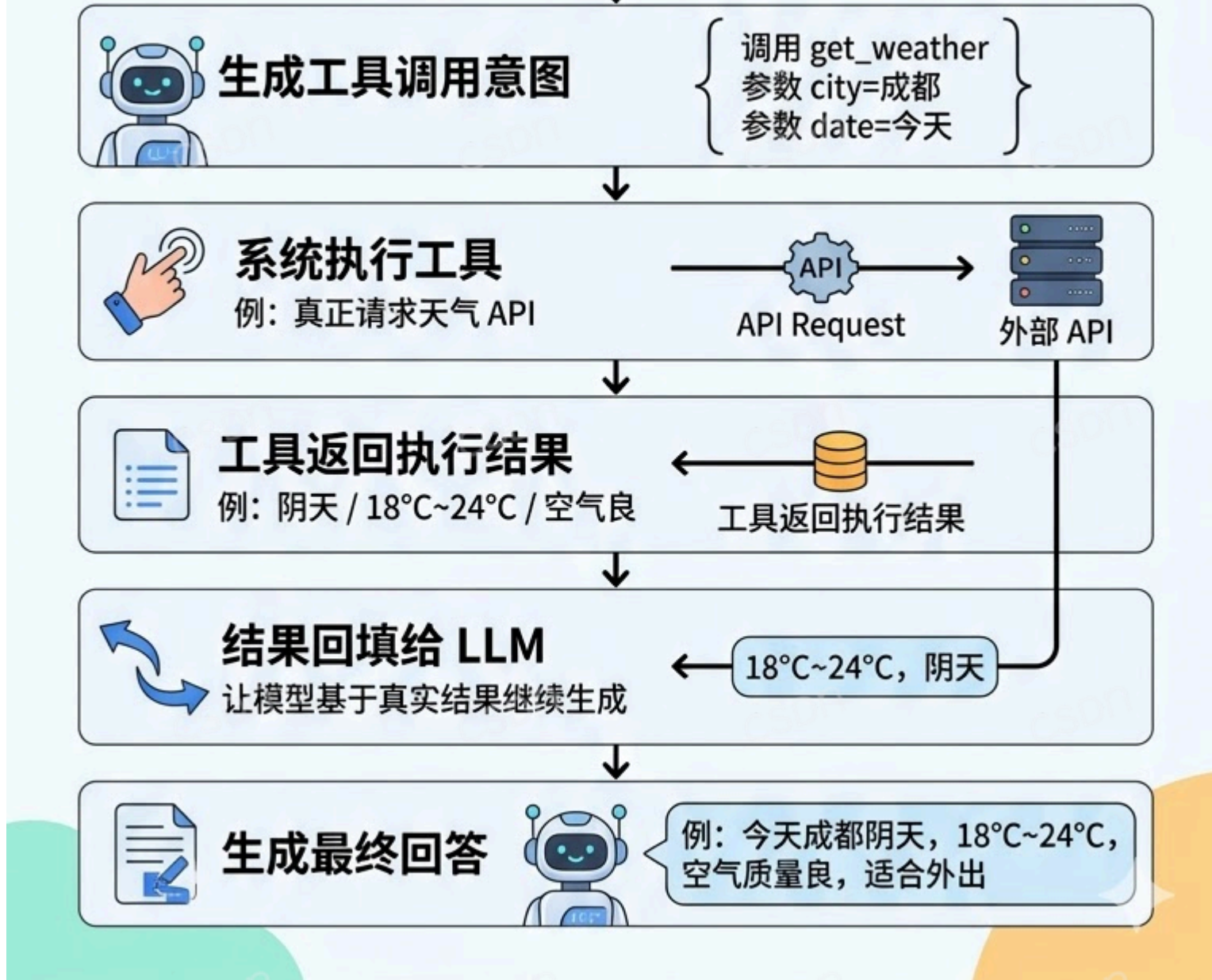
作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159349579

作者主页：https://blog.csdn.net/m0_46279060

LLM 结合外部工具处理用户问题流程





先说结论：

- 系统先把可用工具清单告诉大模型

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159349579

作者主页：https://blog.csdn.net/m0_46279060

- 模型再根据用户问题和工具描述之间的语义匹配，决定要不要调工具、该调哪个工具。

第一步：系统先把工具告诉模型

这份工具清单通常会包含：

- 工具名
- 功能描述
- 参数说明

比如图里就举了一个很典型的例子：`get_weather(city, date)`

这相当于系统先递给模型一份“工具说明书”，告诉它：现在你可以使用天气工具，而且它需要城市和日期两个参数。

关键词是：**不是模型自己发现工具，而是系统先提供工具**

第二步：模型先理解用户到底要什么

工具清单给到模型之后，模型不会立刻选工具。它先要理解用户问题的真实意图。比如图里的案例是：

帮我查一下今天成都天气

这句话表面上只是一个自然语言问题，但模型需要先把它理解成一个任务类型：

内容来源：[csdn.net](https://blog.csdn.net/m0_46279060)

作者昵称：快乐飞猪ing

文章：https://blog.csdn.net/m0_46279060/article/details/159349579

作者主页：https://blog.csdn.net/m0_46279060

- 这是一个“天气查询任务”
- 而且是一个“实时信息查询”
- 不能只靠模型记忆直接回答

这一步很关键，因为工具选择不是从工具开始的，而是从“**需求理解**”开始的。

如果用户问的是：

- 查天气
- 查订单
- 发邮件

那模型就要先分清楚，这到底属于哪一类任务。

关键词：**先识别需求，再谈选工具**

第三步：模型再理解每个工具能做什么

理解完用户意图之后，模型还要回头看一眼：**当前这些工具分别能干什么？**

这里很多人会忽略一个点：模型并不知道工具背后的真实代码逻辑，它主要依赖的是工具描述文本。

比如图里这几个工具就分别对应不同能力：

- 天气工具查天气

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159349579

作者主页：https://blog.csdn.net/m0_46279060

- 订单工具查订单
- 邮件工具发邮件

也就是说，模型对工具的理解，本质上来自“工具说明”。所以，工具描述写得清不清楚，会直接影响模型能不能理解这个工具适合什么场景。

接下来，模型会把两类信息放在一起比较：

- 用户现在想做什么
- 每个工具分别能做什么

然后进行匹配，即：**工具选择的本质，不是认名字，而是做语义匹配。**

图里写的 semantic links，其实就可以理解成：**用户需求** 和 **工具描述** 之间的语义连接。

比如“查今天成都天气”这个需求，和“查询指定城市天气”这个工具描述，语义高度一致，所以模型更容易选中天气工具。

第四步：模型生成调用意图，系统真正执行

当模型完成工具匹配之后，它并不会自己直接去调用 API，它先做的是生成一段结构化的“**工具调用意图**”。

比如图里的例子就是：

- 调用 `get_weather`
- 参数 `city = 成都`

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159349579

作者主页：https://blog.csdn.net/m0_46279060

- 参数 `date = 今天`

这一步很重要，因为这里体现的是：**模型负责决策，系统负责执行。**

也就是说，模型只是在说：“我建议调用这个工具，并且参数这样填。”

真正去请求天气 API、访问外部服务的，是后面的系统，而不是 LLM 本身。

系统执行完成后，会得到一份结果，比如：

```
bash
1 | - 阴天
2 | - 18°C~24°C
3 | - 空气质量良
```

这份结果先回到系统，再准备交给模型继续处理。

第五步：结果回到模型，模型再组织最终回答

很多人会以为工具执行完就结束了，其实还没有。因为工具返回的结果，通常更像**结构化数据**，还不是用户最终看到的**自然语言**。

所以系统还会把这份结果再回填给 LLM，让模型继续生成最后的回答。

比如天气工具返回：

```
bash
1 | - 18°C~24°C
2 | - 阴天
3 |
```

内容来源: csdn.net

AI写代码 |

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159349579

作者主页: https://blog.csdn.net/m0_46279060

模型拿到这些真实结果之后，才会进一步组织成用户更容易读懂的话，比如：

今天成都阴天，18°C~24°C，空气质量良，适合外出。

这也是为什么完整的 Function Call 流程里，LLM 通常会参与两次：

- 第一次负责理解需求和选择工具
- 第二次负责基于工具结果组织最终回答

总结

写到这里，我现在对“大模型怎么选工具”的理解是：**系统先把工具清单交给模型，模型先理解用户需求，再理解每个工具的能力，最后通过语义匹配选出最合适的工具。**

所以，工具选择这件事，本质上讲的是：**需求理解 + 工具理解 + 语义匹配 + 调用决策。**

提问

继续往下追问，请各位同学思考以下两个问题：

如果工具描述写得不清楚，大模型是不是就更容易选错工具？

Tool Schema 到底应该怎么写，模型才更容易理解？

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159349579

作者主页：https://blog.csdn.net/m0_46279060

见我下篇的分享吧

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159349579

作者主页: https://blog.csdn.net/m0_46279060